LEVELS OF TRANPARENCIES IN DDBMS

NAME :- RAGYAM SINHA
ROLL :- 13000123064
DEPT :- COMPUTER SCIENCE&ENGINEERING
SUBJECT :-DBMS
SEM :- 6

LEVELS OF TRANSPARENCY IN DISTRIBUTED DATABASE MANAGEMENT SYSTEMS (DDBMS)

INTRODUCTION TO DDBMS

- A database system where the data is stored across multiple interconnected computers (nodes).
- Appears as a single logical database to the user.
- Manages the distribution, replication, and access to data.
- Why DDBMS?
- Scalability, Reliability, Availability, Performance, Data Sharing.
- The Challenge:
- Managing the inherent complexity of distributed data.
- Transparency aims to hide this complexity from users and applications.

WHAT IS TRANSPARENCY?

CONTENT:

- Definition: The degree to which the distributed nature of a DDBMS is hidden from users and applications.
- Goal: To make a distributed database appear and function like a centralized one.
- Benefits:
- Simplified application development.
- Improved user experience.
- Easier database administration.
- Enhanced system flexibility and evolution.

KEY LEVELS OF TRANSPARENCY

CONTENT:

- DDBMS offers various levels of transparency, each hiding different aspects of distribution.
- We will discuss the following key levels:

1. Location Transparency
2. Replication Transparency
3. Fragmentation Transparency
4. Network Transparency
5. Local Mapping Transparency
6. Naming Transparency
7. Concurrency Transparency
8. Failure Transparency

- Definition: Users do not need to know the physical location of the data.
- How it works: Data can be moved or replicated without affecting user queries or applications.
- Example: A user queries SELECT * FROM Employees WHERE EmpID = 123; without knowing which specific server stores the Employees table or where EmpID = 123 is physically located.
- Analogy: Dialing a phone number without knowing the physical location of the person you're calling. Image:

LOCATION TRANSPARENCY

REPLICATION TRANSPARENCY

CONTENT:

- Definition: Users and applications are unaware that data may be replicated across multiple sites.
- How it works: The DDBMS automatically manages the consistency of replicated data, ensuring all copies are up-to-date.
- Benefits: Increased data availability, fault tolerance, and query performance (by directing queries to the closest replica).
- Example: If the Products table is replicated on servers in New York and London, a user querying Products doesn't know which replica is being accessed. The DDBMS handles the selection and ensures consistency.

FRAGMENTATION TRANSPARENCY

- Definition: Users and applications are unaware that a relation (table) may be divided into fragments (subsets) and stored at different sites.
- Types of Fragmentation:
- Horizontal Fragmentation: Dividing a table into rows (e.g., Employees by DepartmentID).
- Vertical Fragmentation: Dividing a table into columns (e.g., Employees into Emp_Personal_Info and Emp_Job_Info).
- How it works: The DDBMS reconstructs the fragments to present a complete view of the table to the user.
- Example: A Customers table might be horizontally fragmented by region. A query for all customers appears as if from a single table.

NAMING TRANSPAREN

CONTENT:

- Definition: Every data item in the distributed system has a unique name, and users don't need to know its full qualification or location.
- How it works: The DDBMS maintains a global catalog that maps global names to local names and locations.
- Benefits: Prevents naming conflicts and simplifies referencing data.
- Example: An Order table might exist at multiple sites, but the user simply refers to Order without needing to specify SiteA.Order or SiteB.Order.

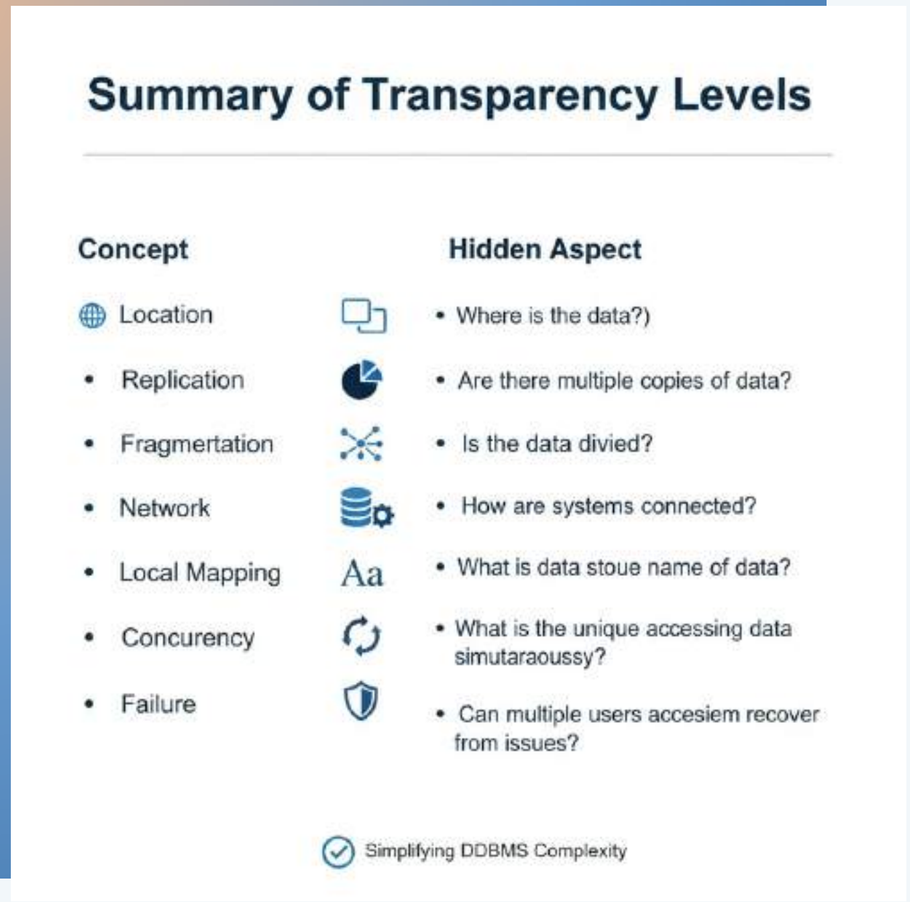
CONCURRENCY TRANSPARENCY

- Definition: Users are unaware of the concurrent execution of transactions by multiple users on shared data.
- How it works: The DDBMS ensures that concurrent transactions execute correctly, preventing anomalies like lost updates or dirty reads, even across multiple sites.
- Mechanisms: Distributed locking, distributed timestamping, optimistic concurrency control.
- Goal: To maintain database consistency and isolation properties in a distributed environment.

FAILURE TRANSPAREN

CONTENT:

- Definition: The DDBMS can recover from failures (e.g., site crashes, network partitions) without affecting ongoing user operations.
- How it works: Uses techniques like replication, distributed commit protocols (e.g., Two-Phase Commit), and logging to ensure data consistency and system availability even in the event of failures.
- Benefits: Increased reliability and availability of the database system.
- Example: If a server fails, transactions in progress are either rolled back or completed on a different replica, and users continue working without interruption.



SUMMARY OF TRANSPARENCY LEVELS

- Location: Where is the data? (Hidden)
- Replication: Are there multiple copies of data? (Hidden)
- Fragmentation: Is the data divided? (Hidden)
- Network: How are systems connected? (Hidden)
- Local Mapping: How is data stored locally? (Hidden)
- Naming: What is the unique name of data? (Simplified)
- Concurrency: Are multiple users accessing data simultaneously? (Managed)
- Failure: Can the system recover from issues? (Handled) Image:

CONCLUSION

The primary objective of transparency in a Distributed Database Management System (DDBMS) is to provide distribution independence. By shielding users and application developers from the underlying complexities—such as where data is stored, how it is partitioned, and how it is replicated—the system allows for a seamless user experience that mimics a centralized database.

Achieving high levels of transparency (especially Fragmentation and Location transparency) significantly reduces the maintenance burden and allows the system to scale or evolve without requiring changes to the application code. Ultimately, these layers of abstraction are what enable DDBMS to offer the benefits of high availability and performance while remaining user-friendly and manageable.

REFERENCES

- *Books:*
- *Principles of Distributed Database Systems by M. Tamer Özsu and Patrick Valduriez.*
- *Database System Concepts by Abraham Silberschatz, Henry F. Korth, and S. Sudarshan.*
- *Distributed Databases: Principles and Systems by Stefano Ceri and Giuseppe Pelagatti.*
- *Academic Resources:*
- *"Introduction to Distributed Databases," IEEE Computer Society.*
- *Lecture Notes on "Distributed DBMS Architecture and Design," various University OpenCourseWare (OCW) portals.*

THANK YOU